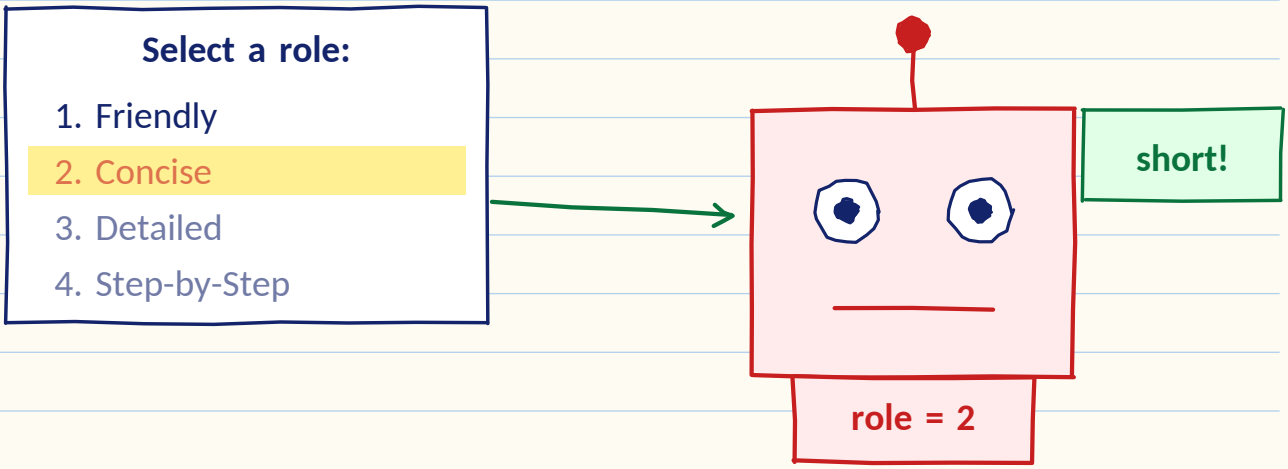
AI Agent Engineering

Day 04

Role Selection: One Assistant, Many Personalities

LangChain + ChatOpenAI + Ollama + Qwen3:1.7B

From intuition and math to complete code logic



roles[choice] -> SystemMessage -> model -> styled answer

Handwritten Study Notes

Author: Chandan Chaudhari

GitHub: chandanc5525

github.com/chandanc5525/AI_Agent_Engineering_Sessions

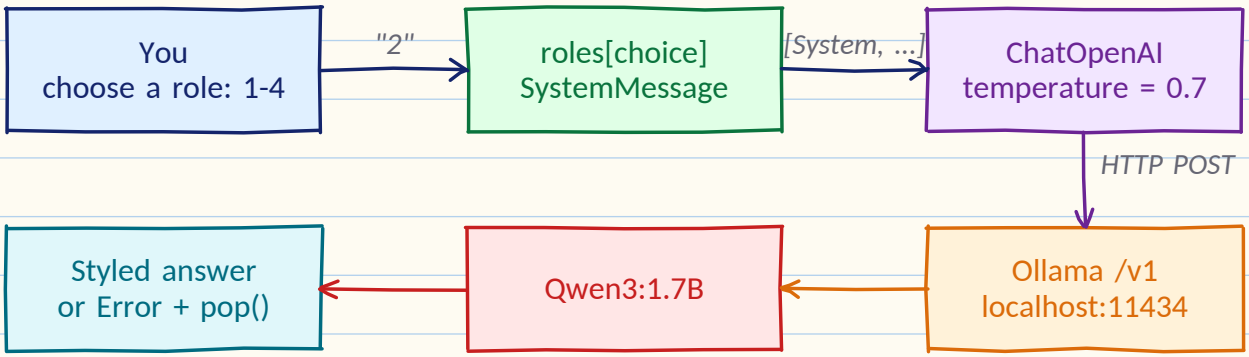
Contents

1. Day 03 Recap and Day 04 Goal 3
2. Roles as Configuration Data 4
3. Dictionaries: Lookup Tables and Keys 6
4. How Style Instructions Steer the Model 7
5. Verbosity Cost: Latency and History Growth 9
6. Context Capacity per Role 11
7. Input Validation and Defensive Menus 12
8. Setup and What Is New 13
9. Code Walkthrough: Full Listing 15
10. Role Dictionaries and the Menu 18
11. Selection, Validation and Seeding the List 19
12. The Loop, try/except and Rollback 20
13. Program Flow and State Trace 22
14. Execution Trace and Output 23
15. Where Exceptions Can Come From 24
16. Review: Pitfalls in My Code 26
17. Better Data Model: One Dict, Base + Style 27
18. Improved Assistant 29
19. Role Router: Auto-Selecting a Role 32
20. Measuring Roles: Sampling and Standard Error 34
21. Streamlit with a Role Selector 36
22. Troubleshooting Guide 38
23. Interview Q and A 39
24. Formula Sheet and Glossary 41
25. Exercises, Checklist, Day 05 Preview 42

1. Day 03 Recap and Day 04 Goal

Day 03 hard-coded one system prompt. Day 04 lets the user choose a role at start-up from a menu. The four roles share the same base sentence and differ only in a style phrase: friendly, concise, detailed or step-by-step. The program stores them in a dictionary, validates the choice, and seeds the history with the chosen SystemMessage.

	Day 03	Day 04
System prompt	One fixed string	Chosen from 4 roles in a dictionary
Start-up	Goes straight to the chat loop	Menu, input, validation, then the loop
Data structures	One string	Two dicts: roles and role_names
Exit check	user_input.lower()	user_input.lower().strip()
On error	Prints the error	Prints the error and calls message.pop()
New idea	System prompt, temperature	Config as data, lookup, validation, rollback



same model, same code path, different system message

After Day 04 I should be able to ...

- store personas as data (dictionaries) and look them up by key
- explain how a small style phrase changes length, latency and history growth, with numbers
- validate user input defensively and choose between "fallback" and "ask again"
- explain why `message.pop()` rolls back correctly only if the `try` wraps the model call alone
- refactor two parallel dicts into one clean data model and measure role behaviour with sampling statistics

One-line summary

Same model, same loop. The role dictionary changes only the first message, and that single message changes the style, speed and cost of every reply.

2. Roles as Configuration Data

A role is a named system prompt. Instead of editing code to change behaviour, I keep roles as data and let the user (or another part of the program) pick one. This is configuration over code, the same idea as `.env`.

Key	Name	Style phrase in the system prompt
"1"	Friendly Assistant	Explain every concept with simple examples
"2"	Concise Assistant	Provide concise answers
"3"	Detailed Assistant	Provide detailed explanations with examples
"4"	Step-by-Step Assistant	Provide step-by-step explanations with examples

All four begin with "You are a helpful assistant." and end with "... code snippets when necessary." Only the middle phrase differs. That is an A/B-test setup: one variable changes, everything else stays fixed.

Why data instead of if/elif?

if/elif chain vs dictionary lookup

```
# fragile: logic repeated, hard to extend
if choice == "1":
    text = "You are a helpful assistant. Explain ..."
elif choice == "2":
    text = "You are a helpful assistant. Provide concise ..."
# ...

# better: data + one lookup
text = roles[choice]
```

- Adding a fifth role is one new dictionary entry, no new branch.
- The menu is generated from the data, so menu and behaviour cannot drift apart.
- Roles can later move to a JSON or YAML file and be edited by non-programmers.

Design principle

Separate what changes (the text of the roles) from what stays (the chat loop). Data changes often; code should not.

3. Dictionaries: Lookup Tables and Keys

A Python dict maps keys to values. My code uses two dicts with the same keys: roles (key to prompt text) and role_names (key to display name).

roles

"1"	You are ... Explain every ...
"2"	You are ... concise answers ...
"3"	You are ... detailed ...
"4"	You are ... step-by-step ...

roles["2"]

role_names

"1"	Friendly Assistant
"2"	Concise Assistant
"3"	Detailed Assistant
"4"	Step-by-Step Assistant

role_names["2"]

Dictionary operations I use

Operation	Example	Result / note
Lookup	roles["2"]	Prompt text; KeyError if the key is missing
Membership	"5" in roles	False (checks keys, not values)
Iterate pairs	for k, v in role_names.items()	Used to print the menu
Safe lookup	roles.get("5", roles["1"])	Default instead of an error

Intuition: why lookups are fast

A dict stores entries in a hash table. Python computes a number from the key (its hash) and jumps to a slot, so lookup does not scan all entries.

slot = hash(key) mod m load factor α = n / m

n entries in m slots; average lookup is O(1), a list search is O(n)

- For 4 roles speed is irrelevant, but the same pattern scales to thousands of entries (for example intents, tools or users).
- Keys here are strings ("1", "2"). input() always returns a string, so comparing with the integer 2 would never match.
- Two parallel dicts must keep identical key sets. If a key exists in roles but

not in role_names, role_names[key] raises KeyError (Section 17 fixes this).

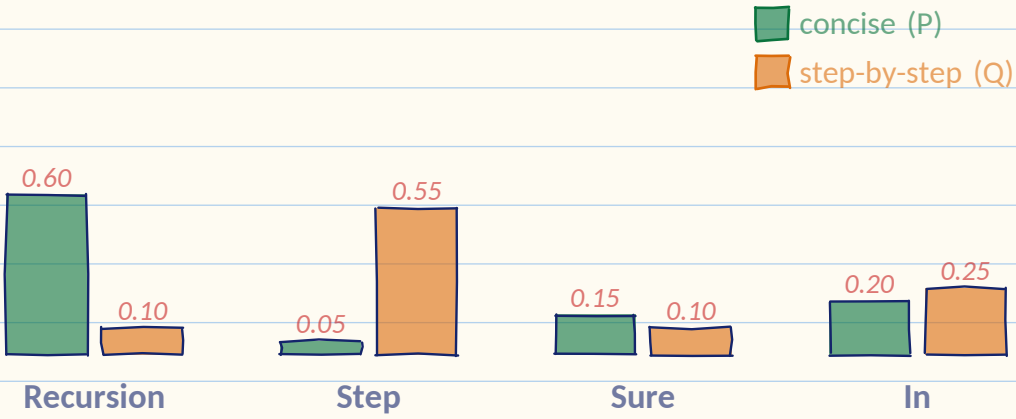
The parallel-dict trap

```
roles = {"1": "...", "2": "..."}
role_names = {"1": "Friendly"}          # key "2" forgotten!

choice = "2"
print(choice in roles)                  # True
print(role_names[choice])               # KeyError: '2'
```

4. How Style Instructions Steer the Model

Why does "Provide concise answers" produce short text? The words in the system prompt are part of the context, so they change the probability of every next token (Day 03). Illustrative first-token probabilities for the question "What is recursion?":



The two roles lead to different distributions over the first token: concise starts with the answer, step-by-step starts with "Step". The size of the difference can be measured with KL divergence:

$$D_{KL}(P \parallel Q) = \Sigma_i \, P_i \cdot \log_2(P_i / Q_i)$$

bits; 0 when P = Q; here 1.401 bits (and 1.666 bits the other way round: KL is not symmetric)

Interpretation: on average a first token drawn from the concise role is 1.4 bits "surprising" to someone who expects the step-by-step role. The roles really are different distributions, not just different wording.

PyTorch: KL divergence in bits

```python
import torch
import torch.nn.functional as F

P = torch.tensor([0.60, 0.05, 0.15, 0.20])   # concise
Q = torch.tensor([0.10, 0.55, 0.10, 0.25])   # step-by-step
# kl_div(input=log Q, target=P) = sum P * (log P - log Q)  (nats)
kl_nats = F.kl_div(Q.log(), P, reduction="sum")
print(kl_nats.item() / torch.log(torch.tensor(2.0)).item())  # 1.401
```

TensorFlow: KL divergence in bits

```python
import tensorflow as tf

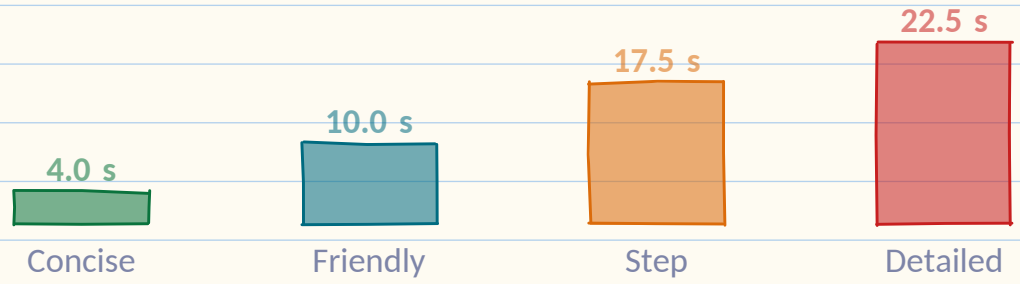
P = tf.constant([0.60, 0.05, 0.15, 0.20])
Q = tf.constant([0.10, 0.55, 0.10, 0.25])
kl = tf.keras.losses.KLDivergence()(P, Q)    # nats, y_true = P
print(float(kl) / float(tf.math.log(2.0)))   # 1.401
```

> **Note**
> The probabilities in this section are illustrative teaching numbers, not measured from Qwen. Measure real behaviour with the sampling method in Section 20.

5. Verbosity Cost: Latency and History Growth

Style phrases change how many tokens the model writes. That has two costs: generation time now, and a longer history for every later turn. Assume (illustrative) average answer lengths and a speed of 20 tokens per second:

Role	Answer a (tokens)	Time = a / 20	t = q + a
Concise	80	4.0 s	130
Friendly	200	10.0 s	250
Step-by-step	350	17.5 s	400
Detailed	450	22.5 s	500



Now the history effect. In Day 03 the total input tokens over n turns, with a system prompt of S tokens and a question of q tokens, was:

$$T_r(n) = n \cdot S + n \cdot q + t_r \cdot n(n - 1) / 2$$

t_r = q + a_r depends on the role; S = 30, q = 50 here

Role	t	T(10) input tokens	vs concise
Concise	130	6,650	1.0×
Friendly	250	12,050	1.8×
Step-by-step	400	18,800	2.8×
Detailed	500	23,300	3.5×

The system prompt S is almost identical across roles; the whole difference comes from the answers stored in history. A verbose role is paid for twice: once when generated and again every later turn when it is resent.

PyTorch: input tokens over 10 turns per role

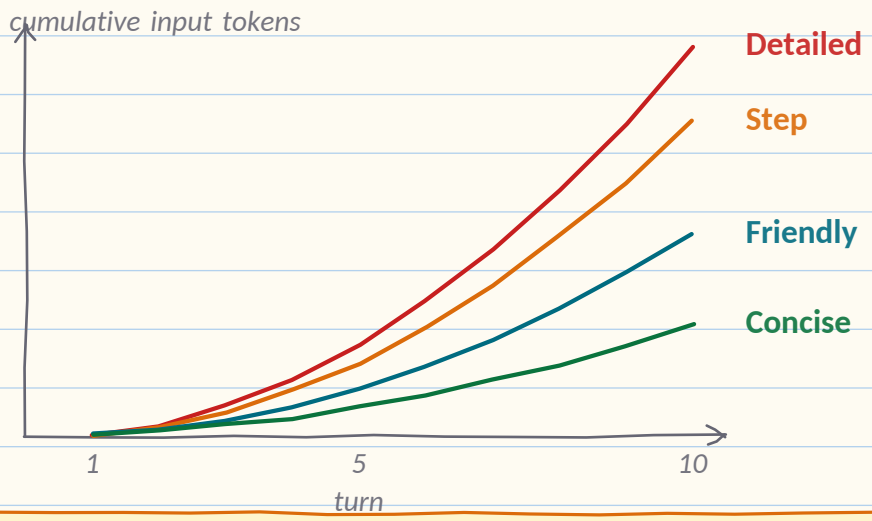
```
import torch

S, q, n = 30, 50, 10
a = torch.tensor([80, 200, 350, 450])         # answer lengths
t = q + a
total = n * S + n * q + t * n * (n - 1) // 2
print(total.tolist())    # [6650, 12050, 18800, 23300]
```

TensorFlow: input tokens over 10 turns per role

```
import tensorflow as tf

S, q, n = 30, 50, 10
a = tf.constant([80, 200, 350, 450])
t = q + a
total = n * S + n * q + t * n * (n - 1) // 2
print(total.numpy())     # [ 6650 12050 18800 23300]
```



Takeaway

"Detailed" costs 3.5 times the tokens of "Concise" over 10 turns with the same question. Choose the role that fits the task, not the longest one.

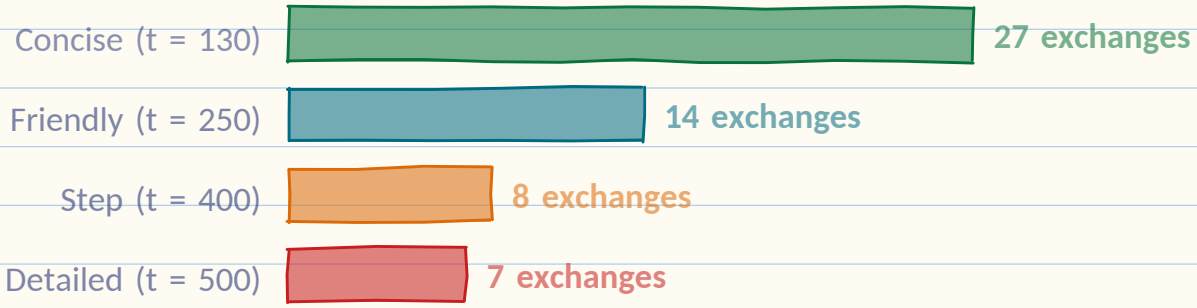
6. Context Capacity per Role

The context window C limits how many exchanges fit. From Day 02: the number of exchanges that fit is

$$k_{max} = \lfloor (C - S - R) / t \rfloor$$

C = window, S = system prompt, R = tokens reserved for the answer, t = tokens per exchange

With C = 4096, S = 30, R = 512 there are 3,554 tokens for history:



Role	t	(4096 - 30 - 512) / t	k_max
Concise	130	3554 / 130 = 27.3	27
Friendly	250	3554 / 250 = 14.2	14
Step-by-step	400	3554 / 400 = 8.9	8
Detailed	500	3554 / 500 = 7.1	7

- A "Detailed" chat forgets its beginning after about 7 exchanges, while a "Concise" one lasts about 27.
- On a small model with a small window, verbose roles trade memory for depth.
- Mitigations: window the history, summarise old turns, or lower the answer length with a limit such as "under 150 words".

Design trade-off

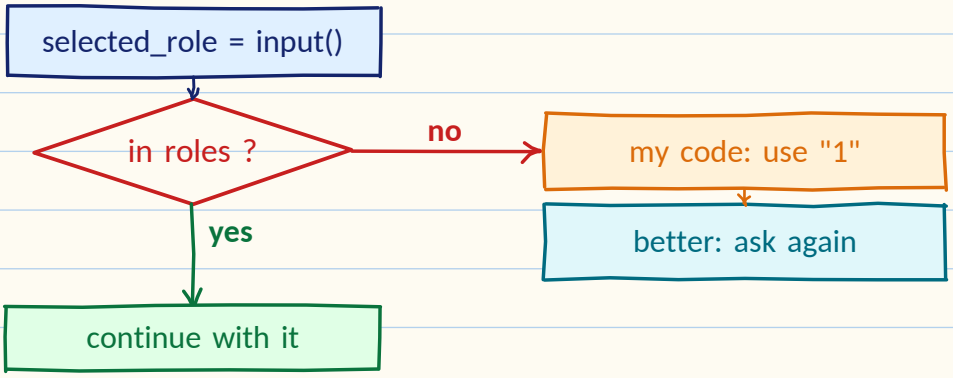
Per-answer quality and per-conversation memory pull in opposite directions. Pick the role (and a length cap) with both in mind.

7. Input Validation and Defensive Menus

The menu reads text typed by a person, so it must be treated as untrusted input. My code checks it with one test:

Validation by fallback

```
if selected_role not in roles:
    print("Invalid selection. Using default role.")
    selected_role = "1"
```



Strategy	Behaviour	Good when
Fallback to default (my code)	Never blocks; silently continues with role 1	Non-interactive use, demos
Re-prompt loop	Repeats the question until valid	Interactive menus
Fail fast	Raise an error and stop	Config files, scripts, tests

Input hygiene checklist

- .strip() removes spaces and newlines: " 2 " becomes "2". My menu does not strip, so " 2" counts as invalid; the exit check does strip.
- .lower() makes "EXIT" equal to "exit"; digits need no case folding.
- Check type and range: here membership in the key set; for numbers use isdigit() and bounds.
- Tell the user what happened ("Invalid selection. Using default role.") so a silent fallback never surprises them.

Re-prompt loop (small, robust)

```
while True:
    pick = input("Enter 1-4: ").strip()
    if pick in roles:
        break
    print("Please type 1, 2, 3 or 4.")
```

> **Rule**
> Never trust input. Validate, normalise (strip, lower), then use. Tell the user when you changed their choice.

8. Setup and What Is New

Setup is identical to Days 02 and 03: langchain-openai, python-dotenv, Ollama running, qwen3:1.7b pulled, and the .env with the /v1 URL.

.env (unchanged)

```
BASE_URL=http://localhost:11434/v1
API_KEY=ollama
MODEL=qwen3:1.7b
```

Terminal

```
uv add langchain-openai python-dotenv
ollama pull qwen3:1.7b
uv run python main.py
```

Day 03 to Day 04: the diff

Change	Why
roles = {...} and role_names = {...}	Personas become data
print menu with for key, value in role_names.items()	Menu generated from the data
selected_role = input(...)	The user chooses before chatting
if selected_role not in roles: ... "1"	Validation with a default
SystemMessage(content=roles[selected_role])	Seed the history with the chosen role
user_input.lower().strip() == "exit"	More forgiving exit check
message.pop() in except	Rollback of the unanswered question

- The model call itself is unchanged: `llm.invoke(message)` with temperature 0.7.
- Everything new happens before the loop (menu) or in the error path (rollback).
- The role is chosen once; to change it you restart the program (Section 18 adds `/role`).

> **Memory aid**
> Day 01 call. Day 02 list. Day 03 system message. Day 04 choose the system message.

9. Code Walkthrough: Full Listing

My complete Day 04 program (long strings split over lines and one prompt shortened, for display only).

main.py (my Day 04 code)

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import (
    HumanMessage, AIMessage, SystemMessage)

load_dotenv()

model_name = os.getenv("MODEL", "qwen3:1.7b")
base_url = os.getenv("BASE_URL", "http://localhost:11434/v1")
api_key = os.getenv("API_KEY", "ollama")

llm = ChatOpenAI(
    model=model_name,
    base_url=base_url,
    api_key=api_key,
    temperature=0.7
)

print("*" * 40)
print("My LangChain Ollama Assistant")
print("*" * 40)

roles = {
    "1": "You are a helpful assistant. Explain every concept "
         "with simple examples and provide code snippets when "
         "necessary.",
    "2": "You are a helpful assistant. Provide concise answers "
         "with code snippets when necessary.",
    "3": "You are a helpful assistant. Provide detailed "
         "explanations with examples and code snippets when "
         "necessary.",
    "4": "You are a helpful assistant. Provide step-by-step "
         "explanations with examples and code snippets when "
         "necessary."
```

```
}

role_names = {
    "1": "Friendly Assistant",
    "2": "Concise Assistant",
    "3": "Detailed Assistant",
    "4": "Step-by-Step Assistant"
}

print("\nSelect a role for the assistant:")

for key, value in role_names.items():
    print(f"{key}. {value}")

selected_role = input("\nEnter the number of your role: ")

if selected_role not in roles:
    print("Invalid selection. Using default role.")
    selected_role = "1"

print(f"\nSelected Role: {role_names[selected_role]}")

message = [
    SystemMessage(content=roles[selected_role])
]

while True:

    user_input = input("\nYou: ")

    if user_input.lower().strip() == "exit":
        print("Exiting the assistant. Goodbye!")
        break

    message.append(HumanMessage(content=user_input))

    try:
        response = llm.invoke(message)
        message.append(AIMessage(content=response.content))
        print("\nAI:", response.content)
```

```
        print("\n" + "=" * 40)
        print("CONVERSATION HISTORY")
        print("=" * 40)

        for msg in message:
            if isinstance(msg, HumanMessage):
                print("\nYou:", msg.content)
            elif isinstance(msg, AIMessage):
                print("\nAI:", msg.content)

        print("=" * 40)

    except Exception as e:
        print("\nError:", e)
        message.pop()
```

Imports and client

- Unchanged from Day 03: ChatOpenAI (OpenAI-compatible client), the three message classes, load_dotenv, os.
- temperature=0.7 applies to all four roles. A concise role may deserve a lower value (Section 17).

10. Role Dictionaries and the Menu

The two dictionaries

Step 1: roles and role_names

```
roles = {
    "1": "You are a helpful assistant. Explain every concept ...",
    "2": "You are a helpful assistant. Provide concise answers ...",
    "3": "You are a helpful assistant. Provide detailed ...",
    "4": "You are a helpful assistant. Provide step-by-step ...",
}

role_names = {
    "1": "Friendly Assistant",
    "2": "Concise Assistant",
    "3": "Detailed Assistant",
    "4": "Step-by-Step Assistant",
}
```

- roles holds the machine text (the system prompt); role_names holds the human label for the menu.
- Both use string keys so they match whatever input() returns.
- The shared prefix "You are a helpful assistant." is repeated four times. If I change it I must change it in four places (DRY violation, fixed in Section 17).
- Triple repetition of "... and provide code snippets when necessary." is also data duplication.

Printing the menu

Step 2: menu from data

```
print("\nSelect a role for the assistant:")

for key, value in role_names.items():
    print(f"{key}. {value}")
```

- role_names.items() yields (key, value) pairs in insertion order (guaranteed since Python 3.7), so the menu prints 1, 2, 3, 4.
- The f-string f"{key}. {value}" inserts the variables: "1. Friendly Assistant".

- Because the menu is built from the dictionary, a new entry appears in the menu automatically.

Output

Select a role for the assistant:1. Friendly Assistant 2. Concise Assistant 3. Detailed Assistant 4. Step-by-Step Assistant

11. Selection, Validation and Seeding the List

Step 3: choose, validate, seed

```
selected_role = input("\nEnter the number of your role: ")

if selected_role not in roles:
    print("Invalid selection. Using default role.")
    selected_role = "1"

print(f"\nSelected Role: {role_names[selected_role]}")

message = [
    SystemMessage(content=roles[selected_role])
]
```

1. input() returns the typed text as a string, including any spaces.
2. selected_role not in roles tests whether the string is one of the dict keys ("1".."4").
3. On failure the program prints a notice and sets the default key "1" (Friendly).
4. role_names[selected_role] is safe now: the key is guaranteed to exist in roles, and both dicts share keys.
5. SystemMessage(content=roles[selected_role]) creates the instruction message from the chosen text.
6. The list literal [SystemMessage(...)] seeds the history: index 0 is the system message for the entire run.

What can go wrong here

Input	Result	Why
2	Concise role	Valid key
" 2" (leading space)	Invalid, falls back to "1"	No strip(); " 2" is not a key
two	Invalid, falls back to "1"	Not a key
(just Enter)	Invalid, falls back to "1"	Empty string is not a key
exit	Invalid, falls back to "1"	The exit command is not recognised here
Ctrl+C	Traceback	KeyboardInterrupt is not an Exception subclass in a try/except block

Invariant

After this block: selected_role is always a valid key, and message has exactly one element, the SystemMessage. List length after n turns is 2n + 1.

12. The Loop, try/except and Rollback

Step 4: loop with rollback

```
while True:

    user_input = input("\nYou: ")

    if user_input.lower().strip() == "exit":
        print("Exiting the assistant. Goodbye!")
        break

    message.append(HumanMessage(content=user_input))

    try:
        response = llm.invoke(message)
        message.append(AIMessage(content=response.content))
        print("\nAI:", response.content)
        ...  # history printout
```

```
    except Exception as e:
        print("\nError:", e)
        message.pop()
```

Walkthrough

1. input() waits for the question. .lower().strip() == "exit" ends the program even for " Exit ".
2. message.append(HumanMessage(...)) records the question before the call, so the model sees it.
3. try: opens a protected block. A failure anywhere inside jumps to except.
4. llm.invoke(message) sends [System, ..., Human] and returns an AIMessage; message.append(AIMessage(...)) stores the answer.
5. The answer and the history printout run only if everything before them succeeded.
6. except Exception as e: prints the error and calls message.pop(), removing the last element.

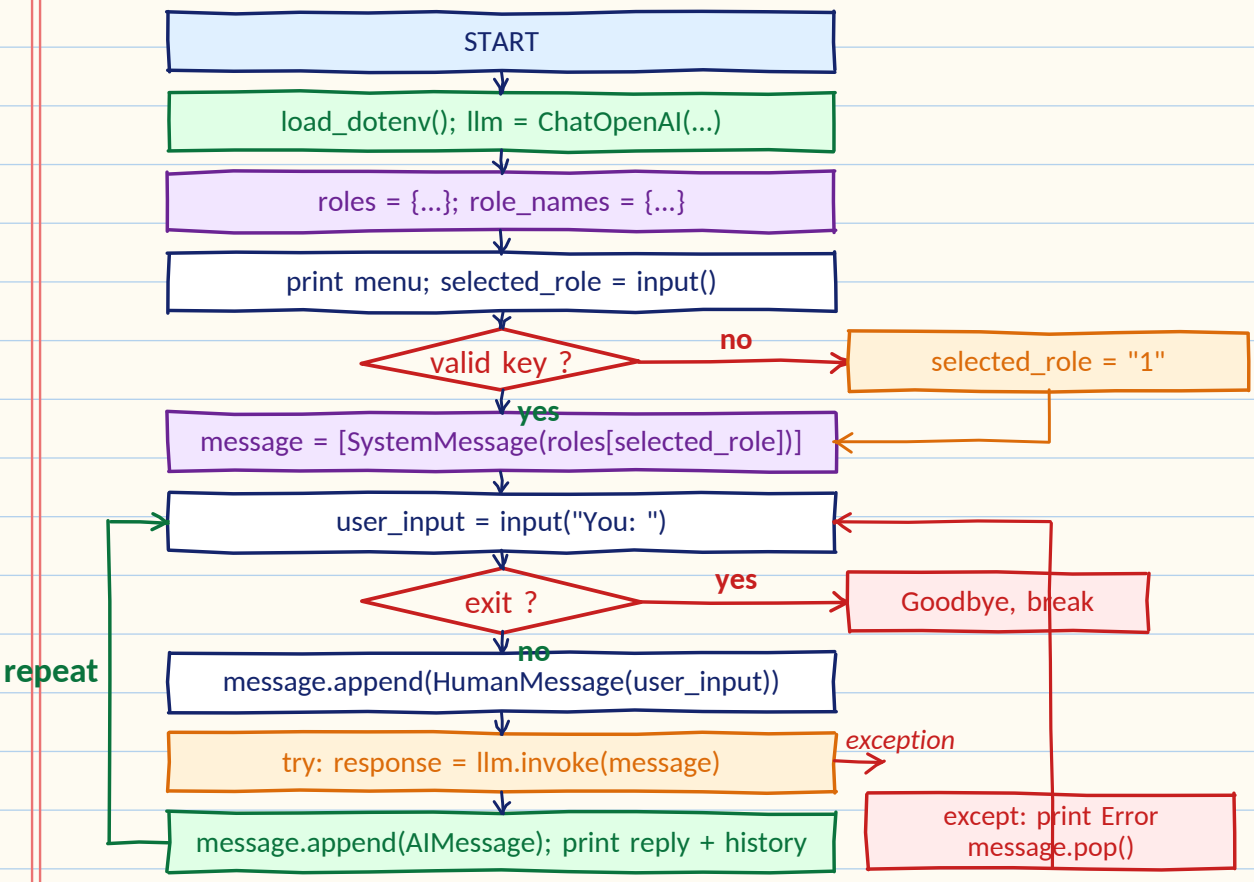
Why pop() is the right idea

Day 03 left the unanswered HumanMessage in the list after an error, so the next call sent two Human messages in a row. pop() removes it, restoring the invariant [System, Human, AI, ..., Human, AI]. This is a rollback: undo a partial change when the operation fails.

Subtle catch

pop() removes the last element, whatever it is. It is correct only if the failure happened before the AIMessage was appended. The try block also wraps the printing, so a failure after the append would remove the wrong message. Section 16 shows this case.

13. Program Flow and State Trace



State trace of the list

S = SystemMessage, H = HumanMessage, A = AIMessage. The rollback keeps the list valid:

start	S	
turn 1 ok	S H1 A1	
turn 2: invoke fails	S H1 A1 H2	pop() -> ok
print fails after append	S H1 A1 H2 A2	pop() hits A2!

14. Execution Trace and Output

Sample session (wording will differ)

```
$ uv run python main.py
****************************************
My LangChain Ollama Assistant
****************************************

Select a role for the assistant:
1. Friendly Assistant
2. Concise Assistant
3. Detailed Assistant
4. Step-by-Step Assistant

Enter the number of your role: 2

Selected Role: Concise Assistant

You: What is a list in Python?

AI: A list is an ordered, mutable collection.
    nums = [1, 2, 3]

========================================
CONVERSATION HISTORY
========================================

You: What is a list in Python?

AI: A list is an ordered, mutable collection.
    nums = [1, 2, 3]
========================================

You: exit
Exiting the assistant. Goodbye!
```

- The menu and "Selected Role" lines appear once; the history block repeats after every turn.
- Invalid choice example: typing 9 prints "Invalid selection. Using default role." and then "Selected Role: Friendly Assistant".

- With Ollama stopped, a question prints `Error: Connection error.`, the message is rolled back, and the next question works once Ollama is back.
- The system message is not printed in the history block, so I cannot see which role is active unless I read the "Selected Role" line.

Experiments to run

1. Ask the same question under all four roles. Count words and time each answer.
2. Type " 2" (with a space) at the menu. What happens? Why?
3. Stop Ollama, ask a question, restart Ollama, ask again. Print `[m.type for m in message]` to check the list.
4. After role 3, ask 8 questions and watch the answers slow down as history grows.

15. Where Exceptions Can Come From

Knowing which line can raise which error tells me where a try block belongs.

Line	May raise	Caught by my except?
input("...") at the menu	EOFError (stdin closed), KeyboardInterrupt (Ctrl+C)	No: outside try (Ctrl+C is not an Exception)
roles[selected_role]	KeyError if key missing (not possible after validation)	No: outside try
role_names[selected_role]	KeyError if dicts diverge	No: outside try
llm.invoke(message)	APIConnectionError, NotFoundError, Timeout, 400/500	Yes
message.append(AIMessage(...))	Practically none	Yes
print("\nAI:", ...)	UnicodeEncodeError on some Windows consoles	Yes, but then pop() hits the wrong element
for msg in message: print(...)	Same as above	Yes, same problem

The rule for placing try

Wrap only the risky operation in try, and keep code that depends on its success in else: (or after it). Then the except block knows exactly what failed, and rollback acts on a known state.

Minimal try, correct rollback

```
message.append(HumanMessage(content=user_input))
try:
    response = llm.invoke(message)       # the only risky call
except Exception as e:
    message.pop()                        # drop the Human msg
    print(f"\nError ({type(e).__name__}):", e)
    continue
message.append(AIMessage(content=response.content))
print("\nAI:", response.content)
```

- continue returns to the top of the loop so the success code does not run.
- type(e).__name__ shows the exception class (ConnectionError vs NotFoundError) instead of only a message.
- Add except KeyboardInterrupt: around the whole program to exit cleanly on Ctrl+C.

Principle

Small try blocks, specific handlers, and rollback that matches exactly what changed.

16. Review: Pitfalls in My Code

The program works and already improves on Day 03. These are the weak spots to know:

#	Issue	Why it matters	Fix
1	try wraps printing; pop() assumes the Human message is last	A print error after the AI append removes the AI message, leaving a dangling Human	Wrap only invoke
2	Menu input is not stripped	" 2" falls back silently to role 1	input().strip()
3	Silent fallback, no re-prompt	User may not notice the wrong role	Re-prompt loop
4	Role fixed for the session	Must restart to change role	/role command
5	Two parallel dicts	Keys can diverge; KeyError	One dict of records
6	Prefix and suffix repeated in all prompts	Edits in four places	BASE + style
7	Same temperature for every role	Concise/step-by-step answers prefer lower values	Per-role temperature
8	No length control	Detailed role may hit the window after ~7 exchanges	Window, word limit
9	History printout every turn	Quadratic noise; hides the system message	Print on command
10	Ctrl+C gives a traceback	KeyboardInterrupt is not an Exception	try/except around main
11	Qwen3 think text may be stored	Reasoning is resent every turn	strip_think()
12	Empty question is sent	Wasted call, odd answers	strip() and skip empties

Seeing issue 1 happen

Wrong element removed

```
msgs = ["S", "H1", "A1", "H2"]       # H2 appended, call succeeded
msgs.append("A2")                    # answer stored
# print(...) raises UnicodeEncodeError here
msgs.pop()                           # except block: removes "A2"!
print(msgs)                  # ['S','H1','A1','H2'] dangling
```

17. Better Data Model: One Dict, Base + Style

Two improvements remove duplication and the chance of mismatched keys: a single dictionary of records, and a shared base + style instead of four full prompts.

before

roles = {"1": "...", ...}

role_names = {"1": "...", ...}

two dicts, keys must match

→

after

ROLES = {

"1": {name, temp, style},

"2": {name, temp, style}, ...}

one record per role

One dict of records + shared base

```
BASE = "You are a helpful assistant."
CODE_RULE = " Add a short code block only when it helps."

ROLES = {
    "1": {"name": "Friendly", "temp": 0.7,
          "style": "Explain every concept with a simple example."},
    "2": {"name": "Concise", "temp": 0.3,
          "style": "Answer in at most 3 sentences."},
    "3": {"name": "Detailed", "temp": 0.5,
          "style": "Give a thorough explanation with examples."},
    "4": {"name": "Step-by-Step", "temp": 0.3,
          "style": "Explain in numbered steps."},
}
```

```
def system_text(key: str) -> str:
    return f"{BASE} {ROLES[key]['style']}{CODE_RULE}"
```

Benefits

- One source of truth: the name, temperature and style of a role live together; no key can be missing in a second dict.
- DRY: the shared base sentence and code rule exist once.
- Per-role settings: concise and step-by-step answers use a lower temperature (more focused), friendly keeps 0.7.
- Easy to extend: add `"max_words"` or `"model"` to the record later; the loop does not change.

The same structure can be written as a dataclass for type safety, or loaded from `roles.json` so non-programmers can edit it:

dataclass version

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Role:
    name: str
    temp: float
    style: str


ROLES = {"1": Role("Friendly", 0.7, "Explain with an example.")}
print(ROLES["1"].name)           # attribute access
```

18. Improved Assistant

Same menu idea with: re-prompt loop, strip, one dict of records, per-role temperature, correct rollback, /role to switch, bounded history, think-block cleaning and a clean main().

main.py (improved Day 04 assistant)

```
import os
import re
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import (
    AIMessage, HumanMessage, SystemMessage)

load_dotenv()

BASE = "You are a helpful assistant."
CODE_RULE = " Add a short code block only when it helps."
ROLES = {
    "1": {"name": "Friendly", "temp": 0.7,
          "style": "Explain every concept with a simple example."},
    "2": {"name": "Concise", "temp": 0.3,
          "style": "Answer in at most 3 sentences."},
    "3": {"name": "Detailed", "temp": 0.5,
          "style": "Give a thorough explanation with examples."},
    "4": {"name": "Step-by-Step", "temp": 0.3,
          "style": "Explain in numbered steps."},
}
MAX_TURNS = 8


def make_llm(temp: float) -> ChatOpenAI:
    return ChatOpenAI(
        model=os.getenv("MODEL", "qwen3:1.7b"),
        base_url=os.getenv("BASE_URL", "http://localhost:11434/v1"),
        api_key=os.getenv("API_KEY", "ollama"),
        temperature=temp,
        timeout=120,
    )
```

```
def choose_role() -> str:
    print("\nSelect a role:")
    for key, role in ROLES.items():
        print(f"{key}. {role['name']}")
    while True:
        pick = input("Enter 1-4 (or q): ").strip().lower()
        if pick == "q":
            raise SystemExit
        if pick in ROLES:
            return pick
        print("Invalid choice, try again.")


def system_message(pick: str) -> SystemMessage:
    text = f"{BASE} {ROLES[pick]['style']}{CODE_RULE}"
    return SystemMessage(content=text)


def strip_think(text: str) -> str:
    pat = r"<think>.*?</think>"
    return re.sub(pat, "", text, flags=re.DOTALL).strip()


def main() -> None:
    pick = choose_role()
    llm = make_llm(ROLES[pick]["temp"])
    history = []
    print(f"Role: {ROLES[pick]['name']}   (/role to change, exit)")
    while True:
        q = input("\nYou: ").strip()
        if q.lower() == "exit":
            print("Goodbye!")
            break
        if q.lower() == "/role":
            pick = choose_role()
            llm = make_llm(ROLES[pick]["temp"])
            history.clear()            # fresh start
            continue
        if not q:
            continue
```

```
        history.append(HumanMessage(content=q))
        try:
            msgs = [system_message(pick)] + history[-2 * MAX_TURNS:]
            reply = llm.invoke(msgs)
        except Exception as err:
            history.pop()               # drop the Human msg
            print(f"\n[error] {type(err).__name__}: {err}")
            continue
        answer = strip_think(reply.content)
        history.append(AIMessage(content=answer))
        print("\nAI:", answer)

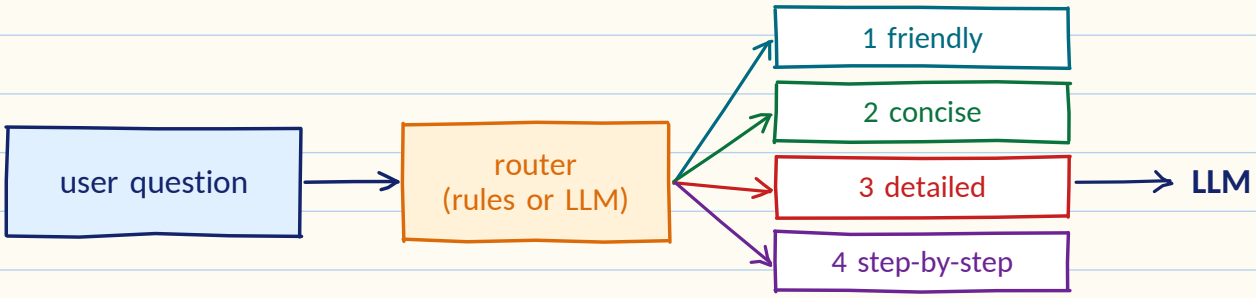

if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, SystemExit):
        print("\nBye!")
```

What changed and why

Change	Reason
choose_role() with while True and strip()	No silent wrong role; forgiving input
ROLES records with name, temp, style	One source of truth; per-role temperature
system_message() builds BASE + style	No duplicated prompt text
try wraps only invoke	pop() always removes the Human message
/role command and history.clear()	Switch role without restart; old style does not leak
history[-2 * MAX_TURNS:]	Bounded cost; system message always added separately
SystemExit and KeyboardInterrupt handling	Clean exit on q or Ctrl+C

19. Role Router: Auto-Selecting a Role

Why make the user choose? A first step toward agent behaviour is letting the program choose the role itself from the question. This is routing: classify the input, then dispatch to the right handler.



Version 1: keyword rules (free and instant)

Rule-based router

```
KEYWORDS = {
    "2": ["quick", "short", "tl;dr", "briefly"],
    "4": ["step", "how do i", "walk me through"],
    "3": ["in detail", "deep dive", "why does"],
}


def route(question: str, default: str = "1") -> str:
    low = question.lower()
    for role, words in KEYWORDS.items():
        if any(w in low for w in words):
            return role
    return default
```

Version 2: ask the model to classify

LLM router

```
def route_llm(question: str, llm) -> str:
    ask = (
        "Reply with ONE digit. 1=friendly, 2=concise, "
        "3=detailed, 4=step-by-step. Best style for:\n" + question
    )
    out = llm.invoke(ask).content.strip()
    return out if out in ROLES else "1"      # validate the output!
```

	Rules	LLM classifier
Cost	Free, microseconds	Extra model call (seconds)
Accuracy	Only matches listed words	Understands meaning, but can be wrong
Predictability	Deterministic	Varies with temperature
Maintenance	Edit keyword lists	Edit the classifier prompt

- Always validate the router output (`out in ROLES`) because the model may reply with extra words.
- Per-question routing changes the system message every turn; this breaks prefix caching and mixes styles in history, so route once per conversation topic if possible.
- This pattern (classify, then dispatch) is the seed of agents with tools: the router later chooses a tool instead of a style.

20. Measuring Roles: Sampling and Standard Error

Because sampling is random (temperature 0.7), one answer proves nothing. To compare roles, sample k runs per role and compare the mean length with its standard error.

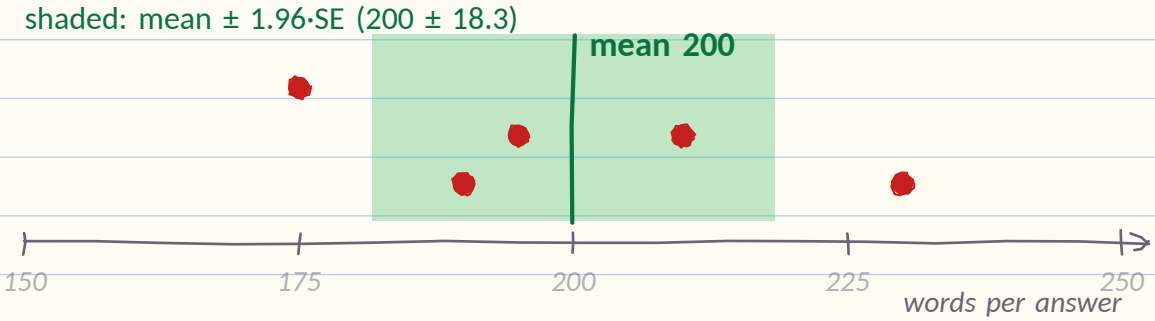
Example: five runs of the Friendly role on one question gave 190, 210, 175, 230 and 195 words.

$$\text{mean } \bar{x} = 200 \quad s = \sqrt{\left(\Sigma (x_i - \bar{x})^2 / (k - 1) \right)} = 20.92$$

sample standard deviation with k - 1 in the denominator (Bessel correction)

$$SE = s / \sqrt{k} = 20.92 / \sqrt{5} = 9.35$$

standard error of the mean; approx. 95% interval: $\bar{x} \pm 1.96 \cdot SE = 200 \pm 18.3$



PyTorch: mean, std, standard error

```
import torch

x = torch.tensor([190.0, 210.0, 175.0, 230.0, 195.0])
mean = x.mean()
s = x.std()                    # unbiased: divides by (k - 1)
se = s / x.numel() ** 0.5
print(mean.item(), round(s.item(), 2), round(se.item(), 2))
# 200.0 20.92 9.35
```

TensorFlow: mean, std, standard error

```
import tensorflow as tf

x = tf.constant([190.0, 210.0, 175.0, 230.0, 195.0])
k = tf.cast(tf.size(x), tf.float32)
mean = tf.reduce_mean(x)
s = tf.sqrt(tf.reduce_sum((x - mean) ** 2) / (k - 1))
se = s / tf.sqrt(k)
print(float(mean), round(float(s), 2), round(float(se), 2))
```

Note: `tf.math.reduce_std` divides by k (population standard deviation), so I compute the sample version by hand.

Harness for the four roles

Compare roles with sampling statistics

```
import statistics as stats


def sample_words(key, question, llm, k=5):
    sizes = []
    for _ in range(k):
        msgs = [system_message(key), HumanMessage(content=question)]
        out = strip_think(llm.invoke(msgs).content)
        sizes.append(len(out.split()))
    return sizes

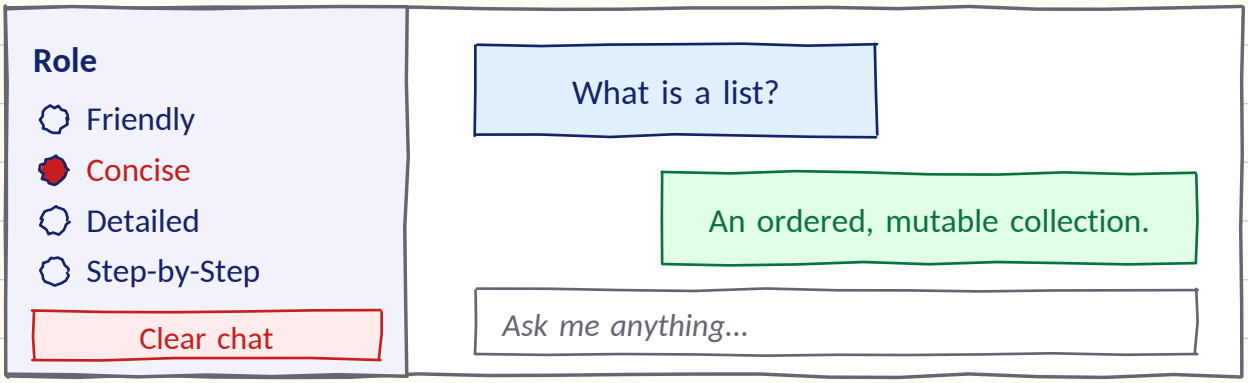

for key in ROLES:
    sz = sample_words(key, "What is recursion?", llm)
    se = stats.stdev(sz) / len(sz) ** 0.5
    name = ROLES[key]["name"]
    print(name, round(stats.mean(sz)), "+/-", round(se, 1))
```

Interpretation

If two role means differ by much more than their standard errors, the difference is real. If the intervals overlap, I cannot tell the roles apart with this many samples.

21. Streamlit with a Role Selector

In Streamlit a radio or selectbox replaces the terminal menu. Because the script reruns on every interaction, the chosen role and the history live in st.session_state, and changing the role should reset the chat. (Reference implementation; not a copy of the repo file.)



Day04/app.py (reference version)

```
import os
import streamlit as st
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import (
    AIMessage, HumanMessage, SystemMessage)

load_dotenv()
BASE = "You are a helpful assistant."
ROLES = {
    "1": {"name": "Friendly", "temp": 0.7,
          "style": "Explain every concept with an example."},
    "2": {"name": "Concise", "temp": 0.3,
          "style": "Answer in at most 3 sentences."},
}


@st.cache_resource
def get_llm(temp: float):
    return ChatOpenAI(
        model=os.getenv("MODEL", "qwen3:1.7b"),
        base_url=os.getenv("BASE_URL", "http://localhost:11434/v1"),
        api_key=os.getenv("API_KEY", "ollama"),
```

```
        temperature=temp,
    )


st.title("Day 04: Role Selector")
key = st.sidebar.radio(
    "Role", list(ROLES), format_func=lambda k: ROLES[k]["name"])
if st.session_state.get("role") != key:
    st.session_state.role = key          # role changed:
    st.session_state.history = []        # start a fresh chat
history = st.session_state.history

for m in history:
    who = "user" if isinstance(m, HumanMessage) else "assistant"
    with st.chat_message(who):
        st.write(m.content)

prompt = st.chat_input("Ask me anything...")
if prompt:
    with st.chat_message("user"):
        st.write(prompt)
    history.append(HumanMessage(content=prompt))
    text = f"{BASE} {ROLES[key]['style']}"
    try:
        reply = get_llm(ROLES[key]["temp"]).invoke(
            [SystemMessage(content=text)] + history)
    except Exception as err:
        history.pop()
        st.error(f"{type(err).__name__}: {err}")
    else:
        history.append(AIMessage(content=reply.content))
        with st.chat_message("assistant"):
            st.write(reply.content)
```

- `st.sidebar.radio(..., format_func=...)` shows friendly names but returns the dict key.
- Comparing `session_state.role` with the new key detects a role change and resets the history.
- `try` / `except` / `else` keeps rollback exact: `else` runs only if `invoke` succeeded.
- The first run has no `session_state.role`, so `.get()` returns None, the condition

is true, and both keys are initialised.

22. Troubleshooting Guide

Symptom	Likely cause	Fix
Always "Friendly Assistant" although I typed 2	Input had a space or newline; fell back to "1"	.strip() and re-prompt loop
KeyError: '2'	role_names missing a key present in roles	One dict of records
Error: Connection error.	Ollama not running	Start Ollama; the rollback keeps the list valid
404 / NotFoundError	BASE_URL missing /v1, or model not pulled	Add /v1; ollama pull qwen3:1.7b
Roles feel the same	Prompts differ by one phrase; small model; high temperature	Make styles explicit (word limits, numbered steps); lower temperature
Detailed role forgets the start	Long answers fill the window	Window, shorter answers, summary
Slow replies in Detailed role	More output tokens and a bigger prompt	Shorter style, fewer turns
Dangling Human message after an error	try wraps printing; pop() removed the wrong item	try around invoke only
Ctrl+C prints a traceback	KeyboardInterrupt is not an Exception	Wrap main in try/except KeyboardInterrupt
Old style leaks after switching role	Previous answers remain in history	history.clear() on switch
Raw think text in answers	Qwen3 reasoning in content	strip_think() or /no_think

Debug checklist

1. Print `selected_role` and `role_names[selected_role]` right after the menu.
2. Print `[m.type for m in message]` before each invoke: system, then alternating human/ai, ending with human.
3. Print `message[0].content` to confirm which prompt is active.
4. Run `ollama run qwen3:1.7b` to check the server and model independently of the code.
5. Compare roles on one fixed question with a fixed seed before judging them.

23. Interview Q and A

Q1. Why store roles in a dictionary?

It separates data from logic: adding or changing a role is a data edit, the menu is generated from it, and lookup by key is O(1).

Q2. Why are the keys strings, not integers?

input() always returns a string, so "2" matches the key "2" while the integer 2 would not.

Q3. What does `selected_role not in roles` check?

Whether the typed string is one of the dictionary keys. Membership tests on a dict look at keys, not values.

Q4. Fallback vs re-prompt for invalid input?

Fallback never blocks but can silently choose the wrong option; re-prompting is better for interactive menus; fail fast suits scripts and config.

Q5. Why does a verbose role cost more than its own answer?

The answers become part of the history and are resent every turn, so total input tokens grow with the per-exchange size t in $n \cdot S + n \cdot q + t \cdot n(n-1)/2$.

Q6. How does the role change the model output?

The system message conditions every next-token distribution, shifting probabilities toward concise, detailed or stepwise text. It does not change the weights.

Q7. What does message.pop() do and when is it correct?

It removes the last list element. It restores the history after a failed call only if the last element is the unanswered Human message, so the try block must wrap only the model call.

Q8. Why is catching Exception broad, and what is better?

It hides the failure type. Catch specific exceptions where possible and print the exception class name; keep a broad handler only at the top level.

Q9. Why does Ctrl+C bypass except Exception?

KeyboardInterrupt derives from BaseException, not Exception, so it must be handled explicitly.

Q10. How would you let users change role mid-chat?

Rebuild the system message and clear (or summarise) the history, because earlier answers in the old style bias the new one. Changing the prefix also invalidates the prompt cache.

Q11. How would you auto-select the role?

Route: keyword rules for speed and predictability, or an LLM classifier for meaning, always validating the output against allowed keys.

Q12. How do you compare two prompts fairly?

Change one thing, sample several runs each, compare means with standard errors ($s/\sqrt{k}$), and keep the question set fixed.

Interview tip

Quote numbers: "Over 10 turns the Detailed role used about 3.5 times the input tokens of Concise, so I cap answer length."

24. Formula Sheet and Glossary

Formula sheet

$$T_r(n) = n \cdot S + n \cdot q + t_r \cdot n(n - 1) / 2$$

total input tokens; t_r = q + a_r; Concise 6,650 vs Detailed 23,300 at n = 10

$$k_{max} = \lfloor (C - S - R) / t \rfloor$$

exchanges that fit in the window

$$time = a / tokens_per_second$$

answer length a; 450 tokens at 20 tok/s = 22.5 s

$$D_{KL}(P \parallel Q) = \Sigma P_i \log_2(P_i / Q_i)$$

how different two token distributions are (bits)

$$s = \sqrt{(\Sigma (x - \bar{x})^2 / (k - 1))}$$

sample standard deviation

$$SE = s / \sqrt{k}$$

standard error of the mean; 95% interval ≈ $\bar{x} \pm 1.96 \cdot SE$

$$slot = hash(key) \bmod m$$

dictionary lookup is O(1) on average

$$list\ length = 2n + 1$$

after n turns: 1 system + n human + n AI messages

Glossary

Term	Meaning
Role / persona	A named system prompt that sets style and behaviour
dict	Python mapping from keys to values; O(1) average lookup
Membership test	`key in dict` checks the keys
Validation	Checking that input is acceptable before using it
Fallback	Default value used when input is invalid
Re-prompt	Asking again until the input is valid
Rollback	Undoing a partial change after a failure
DRY	Do not repeat yourself: one source of truth
KL divergence	Asymmetric measure of difference between two distributions
Standard error	Uncertainty of a sample mean: s / √k
Routing	Classifying input to choose a handler or role
BaseException	Root of exceptions; includes KeyboardInterrupt and SystemExit

25. Exercises, Checklist, Day 05 Preview

Practice exercises

1. Ask one question under all four roles; record word count and time. Do they match the illustrative table in Section 5?
2. Compute T(10) and k_max for your own measured answer lengths using the PyTorch or TensorFlow snippet.
3. Type " 2" at the menu to reproduce the silent fallback, then add `.strip()` and a re-prompt loop.
4. Reproduce the wrong-element pop() bug with the 5-line list demo, then fix it by narrowing the try block.
5. Merge `roles` and `role_names` into one dict of records and add a per-role temperature.
6. Add the `/role` command and verify that history is cleared on a switch.

7. Implement the keyword router and test it on 10 questions; count wrong routes.
8. Run the sampling harness (k = 5) for all four roles and report mean ± SE.
9. Build the Streamlit role selector and confirm that switching resets the chat.

Day 04 completion checklist

[ ] roles and role_names understood as data, with keys as strings
[ ] Menu generated from the dictionary
[ ] I can explain fallback vs re-prompt validation
[ ] message.pop() rollback explained, including the wrong-element case
[ ] I can compute T(n) and k_max per role
[ ] One-dict refactor with per-role temperature done
[ ] Role switching and routing sketched
[ ] Sampling statistics (mean, s, SE) understood
[ ] Streamlit role selector running

Day 05 preview

My expectation (check the repo README): the assistant now has memory, instructions and selectable roles, but it can only talk. The next step is to let it act: call a function or tool (calculator, search, file reader) and use the result. The router from Section 19 becomes a tool chooser.

Final takeaway

Day 01 called the model, Day 02 added memory, Day 03 added control, Day 04 makes that control selectable data. One small system message decides style, speed, cost and how long the conversation can last; the rest of the program stays the same.

- Chandan Chaudhari | github.com/chandanc5525 -